

# Submission Worksheet

## Submission Data

**Course:** IT202-010-S2026

**Assignment:** IT202 Milestone 1 - 2026

**Student:** Joshua T. (jlt36)

**Status:** Submitted | **Worksheet Progress:** 100%

**Potential Grade:** 10.00/10.00 (100.00%)

**Received Grade:** 0.00/10.00 (0.00%)

**Started:** 4/6/2026 1:26:23 AM

**Updated:** 4/8/2026 11:10:19 PM

**Grading Link:** <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-1-2026/grading/jlt36>

**View Link:** <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-1-2026/view/jlt36>

## Instructions

- Overview Link: <https://youtu.be/IDtqdaLwcVg>
- 1. Refer to Milestone1 of this doc: <https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>
- 2. Ensure you read all instructions and objectives before starting.
- 3. Ensure you've gone through each lesson related to this Milestone
- 4. Switch to the Milestone1 branch
  - 1. `git checkout Milestone1` (ensure proper starting branch)
  - 2. `git pull origin Milestone1` (ensure history is up to date)
- 5. Fill out the below worksheet
  - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
  - Ensure proper styling is applied to each page
  - Ensure there are no visible technical errors; only user-friendly messages are allowed
- 6. Once finished, click "Submit and Export"
- 7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
  - 1. `git add .`
  - 2. `git commit -m "adding PDF"`
  - 3. `git push origin Milestone1`
  - 4. On Github merge the pull request from Milestone1 to qa
  - 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the heroku prod links work)
- 8. Upload the same PDF to Canvas
- 9. Sync Local
- 10. `git checkout qa`
- 11. `git pull origin qa`

## Section #1: ( 2 pts.) Feature: User Will Be Able To Register A New Account

Progress: 100%

## ≡ Task #1 ( 0.67 pts.) - Validation

Progress: 100%

### Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

## ≡ Task #1 ( 0.33 pts.) - HTML Form/Validation

Progress: 100%

### Details:

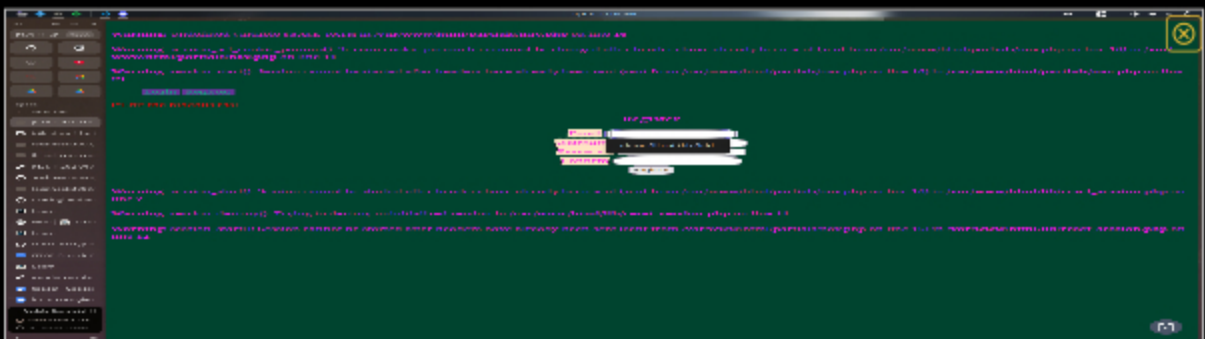
- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

### Part 1:

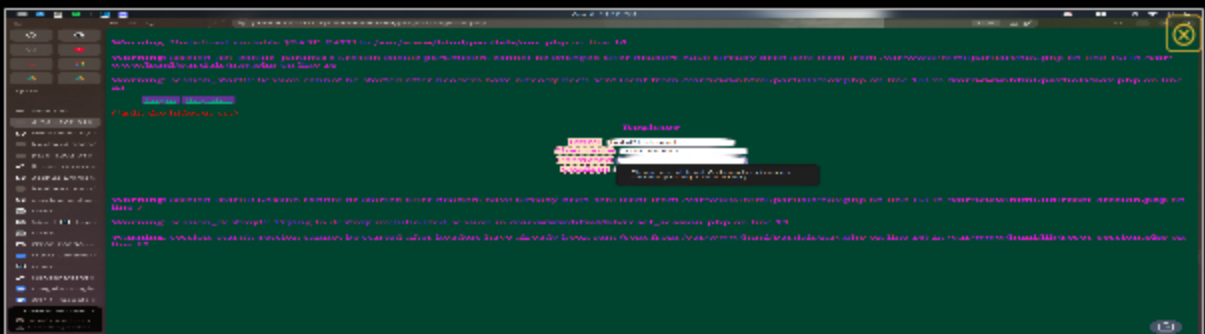
Progress: 100%

### Details:

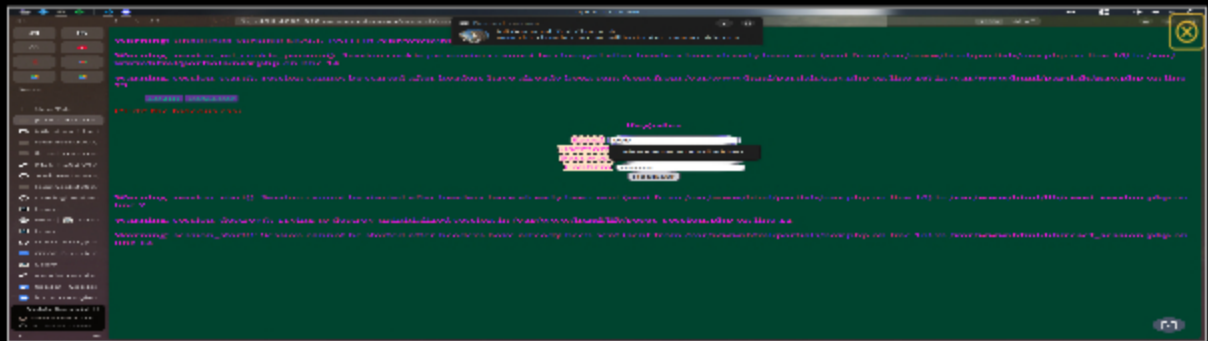
- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)



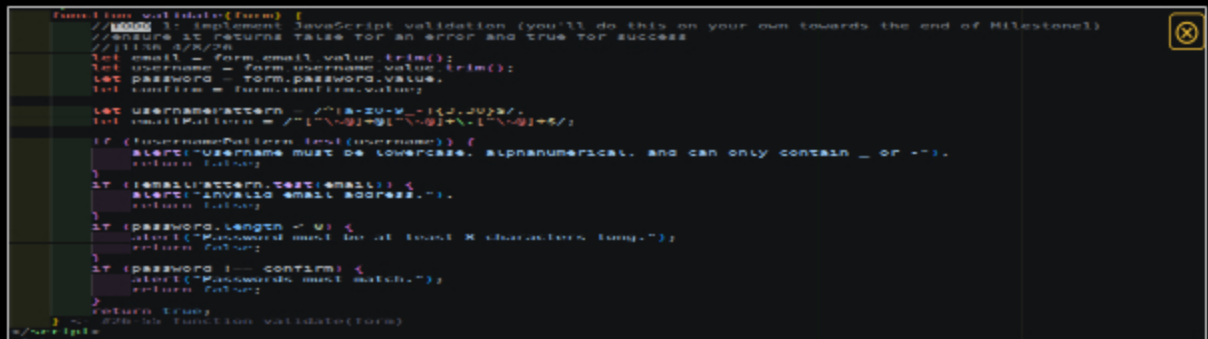
empty field



password must be 8



invalid email address



html validation code snippet

Saved: 4/8/2026 10:51:44 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The register form includes built-in HTML validation attributes such as required, type="email", maxlength, and minlength. These allow the browser to catch missing or improperly formatted input before the form is submitted, This meets the requirement by providing immediate feedback for common errors like mismatching passwords, invalid email formats, and length issues.

Saved: 4/8/2026 10:51:44 PM

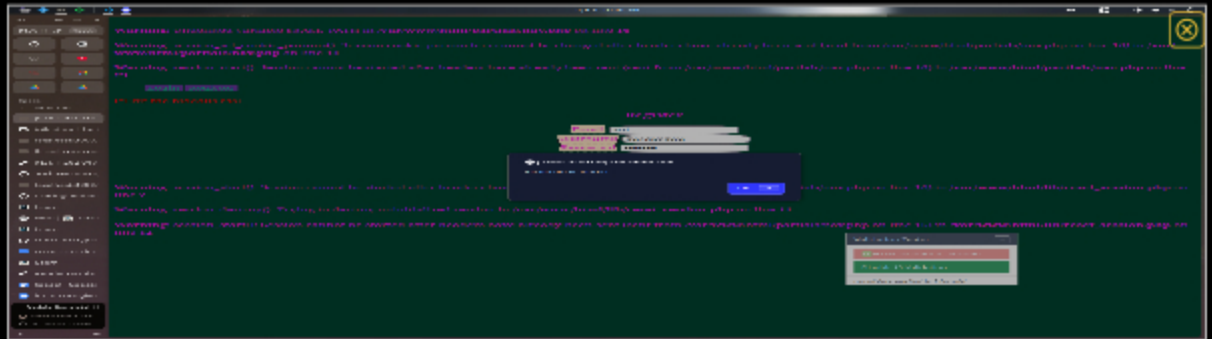
Task #2 ( 0.33 pts.) - JS Validation (validate() function)

Progress: 100%

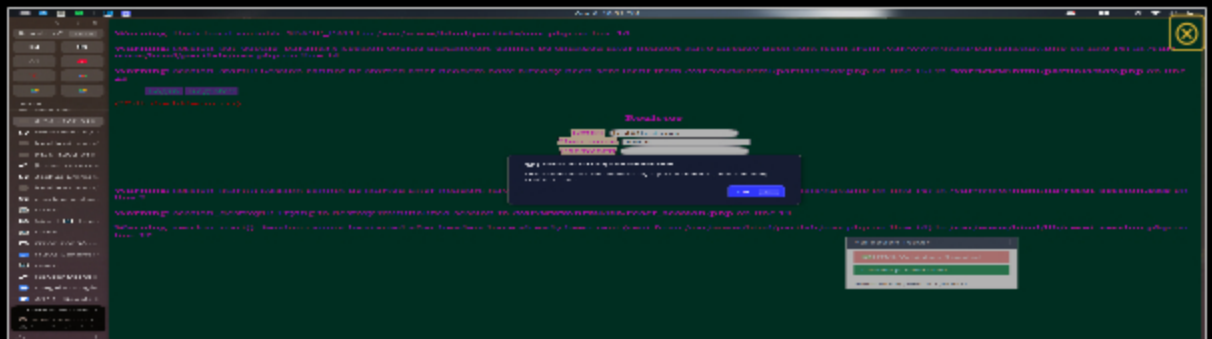
Part 1:

**Details:**

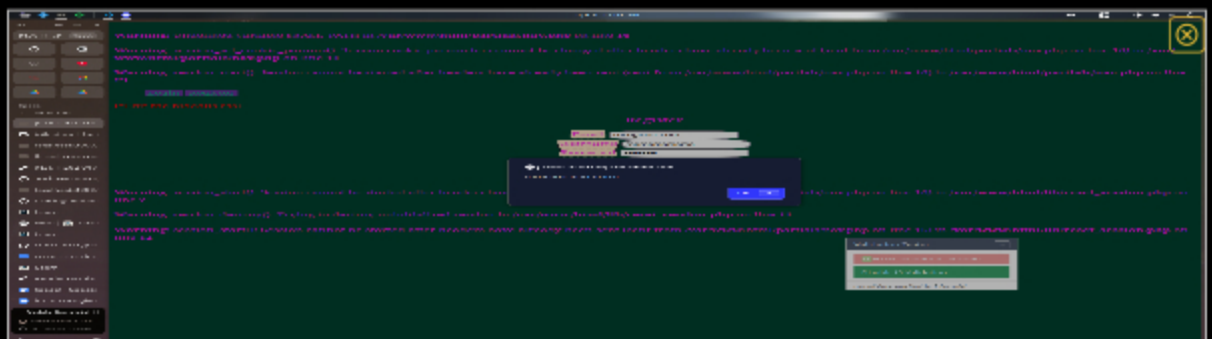
- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply noValidate to the form tag



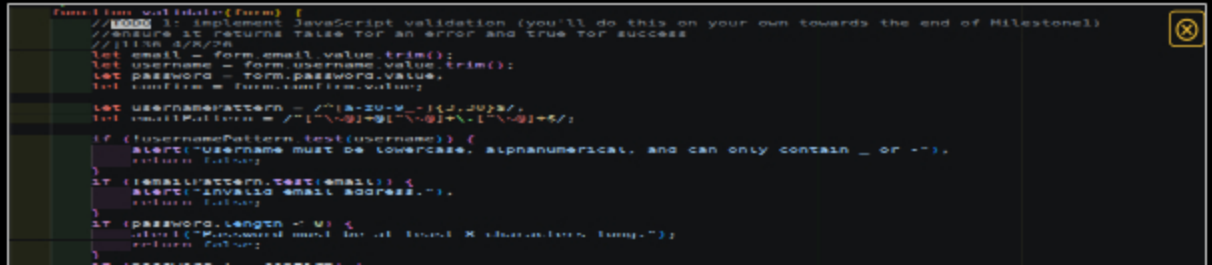
invalid email



username must be lowercase



passwords must match





```
1 // validate email format
2 function validateEmail(email) {
3     const re = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
4     return re.test(email);
5 }
6
7 // validate password strength
8 function validatePassword(password) {
9     const re = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*^&])[a-zA-Z\d@$!%*^&]{8,}$/;
10    return re.test(password);
11 }
12
13 // validate form
14 function validateForm() {
15     const username = document.getElementById('username').value;
16     const email = document.getElementById('email').value;
17     const password = document.getElementById('password').value;
18     const confirmPassword = document.getElementById('confirm-password').value;
19
20     if (!validateEmail(email) || !validatePassword(password) || password !== confirmPassword) {
21         document.getElementById('error-message').innerHTML = 'Invalid email, password, or confirm password. Please try again.';
22         return false;
23     }
24
25     // If all validations pass, proceed with registration logic
26     // ...
27 }
```

js validation code snippet



Saved: 4/8/2026 10:52:06 PM

## Part 2:

Progress: 100%

### Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

### Your Response:

The JavaScript validation step checks the register form before it is submitted to the server. It verifies that the username follows the required format and that the confirm password field matches the password. These checks help meet the requirement by giving fast client-side feedback and preventing invalid form data from being submitted.



Saved: 4/8/2026 10:52:06 PM

## Task #3 ( 0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

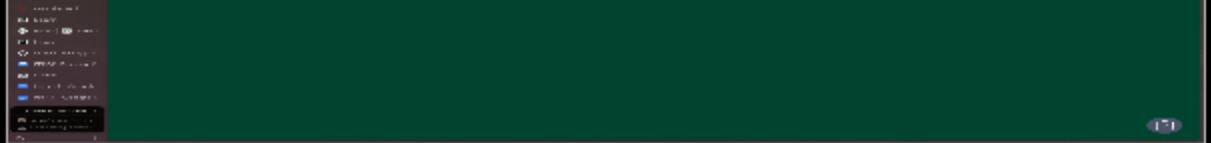
## Part 1:

Progress: 100%

### Details:

- Show the code related to the register page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use





every validation

```
1 // validate.php
2
3 // validate.php MUST NOT BE EMPTY.
4
5 // validate.php MUST NOT BE EMPTY.
6
7 // validate.php MUST NOT BE EMPTY.
8
9 // validate.php MUST NOT BE EMPTY.
10
11 // validate.php MUST NOT BE EMPTY.
12
13 // validate.php MUST NOT BE EMPTY.
14
15 // validate.php MUST NOT BE EMPTY.
16
17 // validate.php MUST NOT BE EMPTY.
18
19 // validate.php MUST NOT BE EMPTY.
20
21 // validate.php MUST NOT BE EMPTY.
22
23 // validate.php MUST NOT BE EMPTY.
24
25 // validate.php MUST NOT BE EMPTY.
26
27 // validate.php MUST NOT BE EMPTY.
28
29 // validate.php MUST NOT BE EMPTY.
30
31 // validate.php MUST NOT BE EMPTY.
32
33 // validate.php MUST NOT BE EMPTY.
34
35 // validate.php MUST NOT BE EMPTY.
36
37 // validate.php MUST NOT BE EMPTY.
38
39 // validate.php MUST NOT BE EMPTY.
40
41 // validate.php MUST NOT BE EMPTY.
42
43 // validate.php MUST NOT BE EMPTY.
44
45 // validate.php MUST NOT BE EMPTY.
46
47 // validate.php MUST NOT BE EMPTY.
48
49 // validate.php MUST NOT BE EMPTY.
50
51 // validate.php MUST NOT BE EMPTY.
52
53 // validate.php MUST NOT BE EMPTY.
54
55 // validate.php MUST NOT BE EMPTY.
56
57 // validate.php MUST NOT BE EMPTY.
58
59 // validate.php MUST NOT BE EMPTY.
60
61 // validate.php MUST NOT BE EMPTY.
62
63 // validate.php MUST NOT BE EMPTY.
64
65 // validate.php MUST NOT BE EMPTY.
66
67 // validate.php MUST NOT BE EMPTY.
68
69 // validate.php MUST NOT BE EMPTY.
70
71 // validate.php MUST NOT BE EMPTY.
72
73 // validate.php MUST NOT BE EMPTY.
74
75 // validate.php MUST NOT BE EMPTY.
76
77 // validate.php MUST NOT BE EMPTY.
78
79 // validate.php MUST NOT BE EMPTY.
80
81 // validate.php MUST NOT BE EMPTY.
82
83 // validate.php MUST NOT BE EMPTY.
84
85 // validate.php MUST NOT BE EMPTY.
86
87 // validate.php MUST NOT BE EMPTY.
88
89 // validate.php MUST NOT BE EMPTY.
90
91 // validate.php MUST NOT BE EMPTY.
92
93 // validate.php MUST NOT BE EMPTY.
94
95 // validate.php MUST NOT BE EMPTY.
96
97 // validate.php MUST NOT BE EMPTY.
98
99 // validate.php MUST NOT BE EMPTY.
100
```

php validation code snippet

 Saved: 4/8/2026 10:53:09 PM

## Part 2:

Progress: 100%

### Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

PHP validation in the register page is handled after the form is submitted using the `$_POST` data. it checks that the email, username, password, and confirm password are not empty, validates the email and username format, ensures the password is at least 8 characters long, and confirms that both password fields match before inserting the user into the database.

 Saved: 4/8/2026 10:53:09 PM

## Task #2 ( 0.67 pts.) - Related URLs

Progress: 100%

### Details:

- Include the direct link to this file from the Milestone branch (should end in `.php`, , zero credit if it does not)
- Include the render.com prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

[https://github.com/Joshua-Emman/jlt36-IT202-010/blob/Milestone1/public\\_html/project/register.php](https://github.com/Joshua-Emman/jlt36-IT202-010/blob/Milestone1/public_html/project/register.php)



URL

<https://github.com/Joshua-Ei>



URL #2

<https://jlt36-it202-010-prod.onrender.com/project/register.php>



URL

<https://jlt36-it202-010-prod.onrender.com/project/register.php>



URL #3

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/21>



URL

<https://github.com/Joshua-Ei>



Saved: 4/6/2026 4:45:31 PM

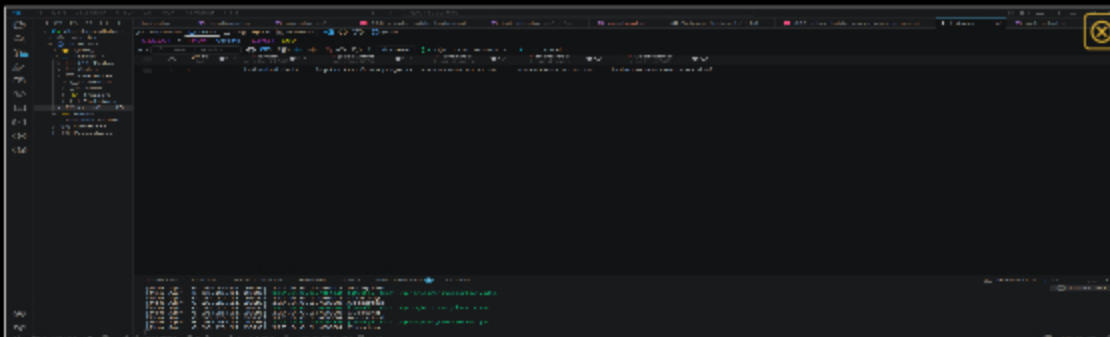


## Task #3 ( 0.67 pts.) - Database - Users table

Progress: 100%

### Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capturing)



bob as my registered user



Saved: 4/6/2026 4:46:19 PM

## Section #2: ( 2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 100%



## Task #1 ( 0.67 pts.) - Validation

Progress: 100%

### Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

## Task #1 ( 0.33 pts.) - HTML Form/Validation

Progress: 100%

### Details:

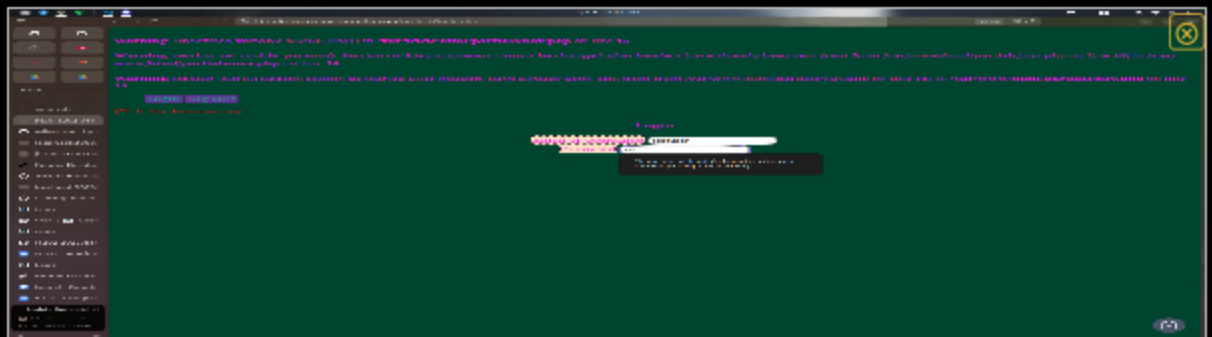
- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

### Part 1:

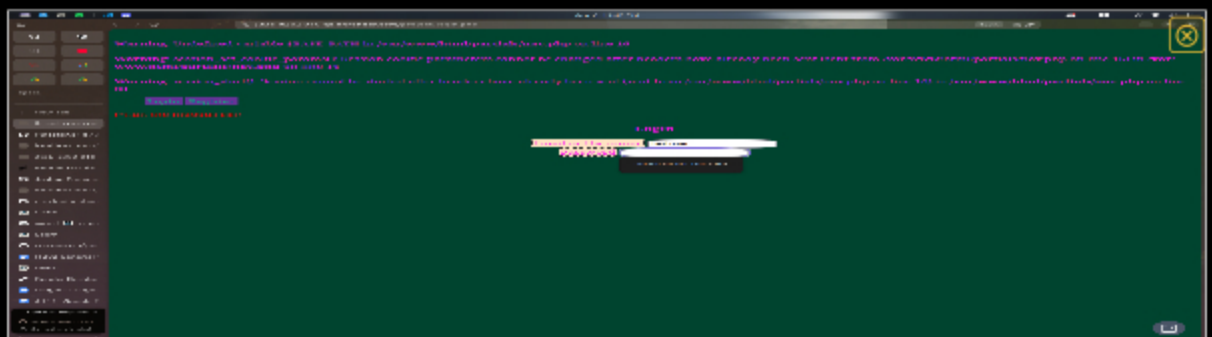
Progress: 100%

### Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)



please use at least 8 characters



empty field

```
<h3>Login</h3>
//jlt36 4/8/26
<form onsubmit="return validate(this)" method="POST">
  <div>
    <label for="email">Email or Username</label>
    <input id="email" type="text" name="email" required value="<?php se($_POST, 'email'); ?>" />
  </div>
</div>
```

```

<label for="pw">Password</label>
<input type="password" id="pw" name="password" required minlength="8" />
</div>
josh, 5 hours ago • trying to fix it again...
<input type="submit" value="Login" />
</form>
<script>

```

html validation code snippet

 Saved: 4/8/2026 10:59:02 PM

## ⇒ Part 2:

Progress: 100%

### Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

### Your Response:

The login form includes built-in HTML validation attributes such as `required` and `minlength` to catch missing or incomplete input before submission. This satisfies the requirement by allowing the browser to immediately show validation messages for empty required fields and passwords that do not meet the minimum length.

 Saved: 4/8/2026 10:59:02 PM

## ≡ Task #2 ( 0.33 pts.) - JS Validation (validate() function)

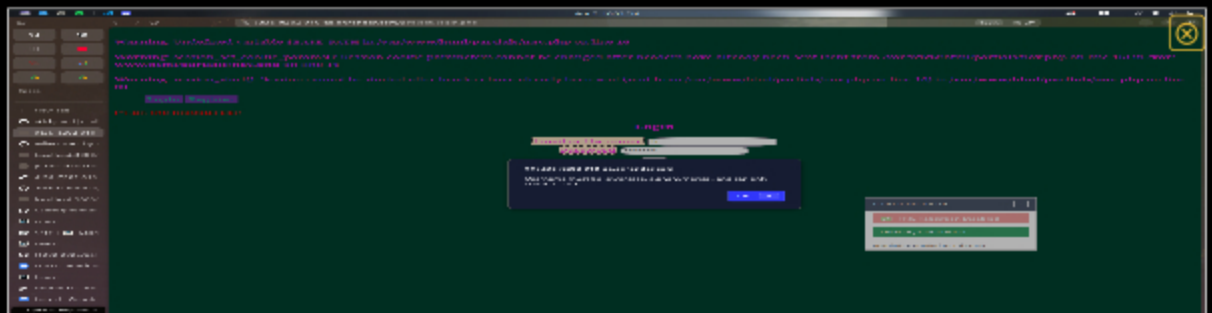
Progress: 100%

## 📄 Part 1:

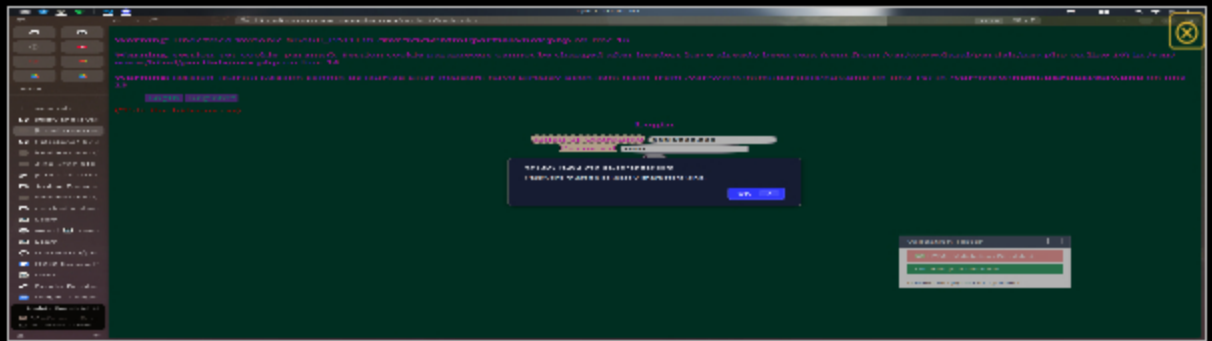
Progress: 100%

### Details:

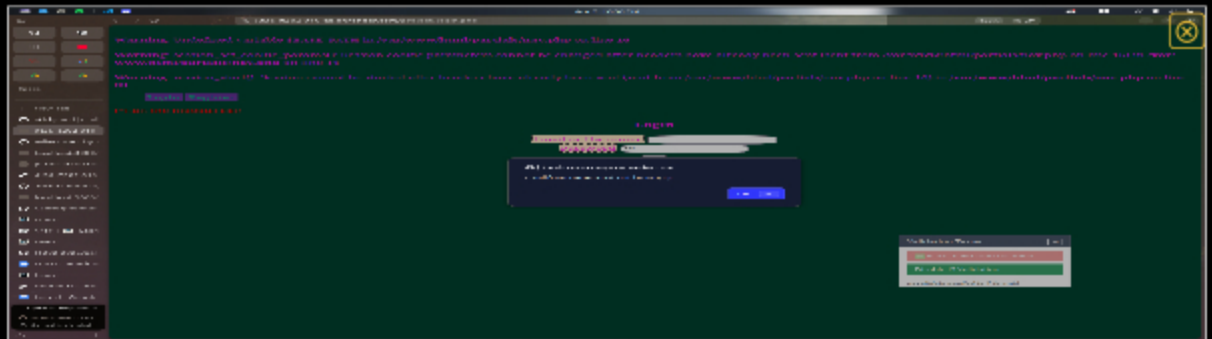
- Show the code related to the login page's `validate()` function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



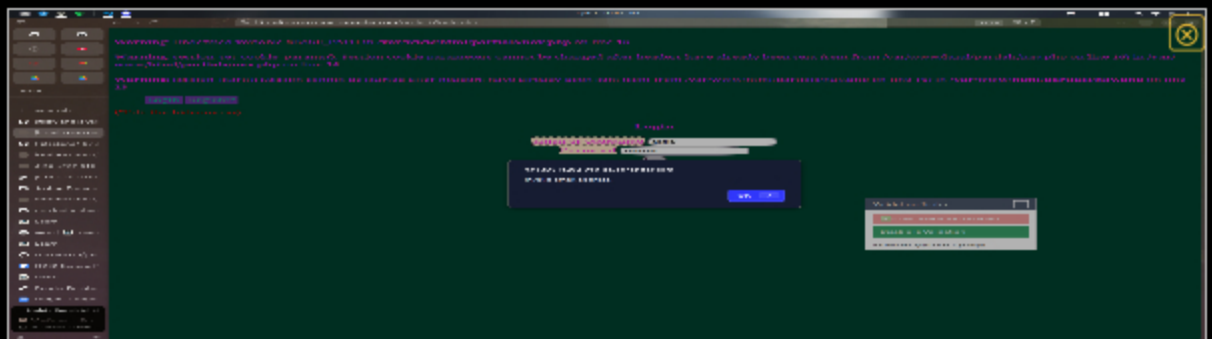
username must be lowercase



password must be 8 characters long



username/email should not be empty



invalid email address



js validation code snippet



**Details:**

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

**Your Response:**

The js validation step checks the login form before it is submitted to the server. It validates whether the entered value is a properly formatted email or username, checks that required fields are not empty, and ensures the password meets the minimum length requirement. This satisfies the requirement by providing immediate client-side feedback before the form is submitted.



Saved: 4/8/2026 10:59:23 PM

## Task #3 ( 0.33 pts.) - PHP Validation (steps before the DB call)

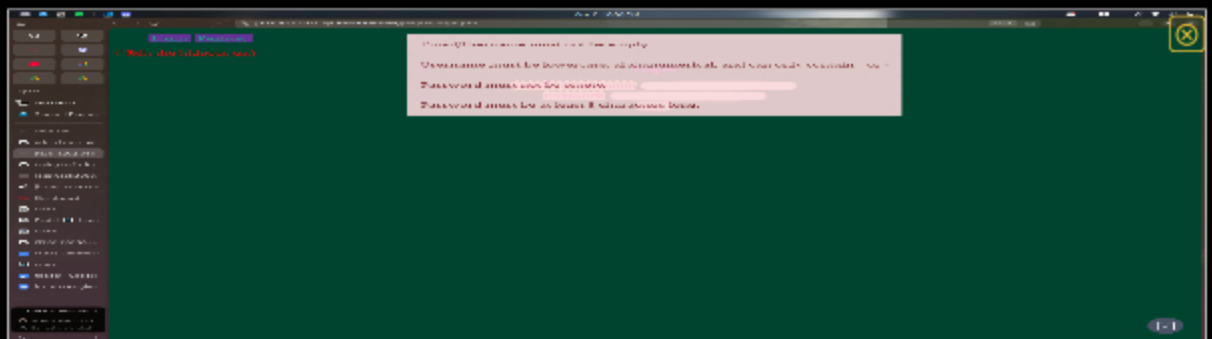
Progress: 100%

**Part 1:**

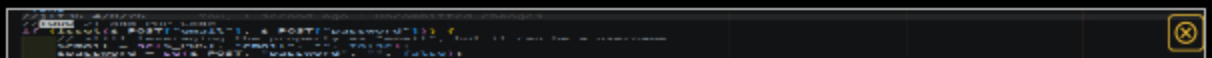
Progress: 100%

**Details:**

- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch (the one that checks against the DB hash)



every validation





[project/login.php](#)

URL #3

[https://github.com/Joshua-Emman/jlt36-](https://github.com/Joshua-Emman/jlt36-IT202-010/)

[IT202-010/](#)

[commit/7999ddd1a835770f1936913fc9140ed13cd5f080](#)



URL

<https://github.com/Joshua-Ei>



Saved: 4/7/2026 11:27:52 PM



## Task #3 ( 0.67 pts.) - User Session Evidence

Progress: 100%

### Details:

- From the render.com logs (Logs tab), capture the session error\_log() output
- It should show the id, username, email, and roles

```
0 02:45:41 AM [prod] [Wed Apr 03 02:45:41 PST 2025] [php notice] [pid 45:file 40] [client 111.0.0.0] Session: array (/\n  user => in array (/\n  'id' => 1/\n  'email' =>\n  'password' => 'password' /\n  'username' => 'bet-8807007-4e032b70-4f0f0c5d' /\n  'roles' => in array (/\n  'id' => in array (/\n  'name' => 'Admin' /\n  ) /\n ), referer: https://jlt36-010-qa-render.com/project/login.php
```

logs



Saved: 4/8/2026 2:46:55 AM

## Section #3: ( 1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

### Task #1 ( 1 pt.) - Evidence

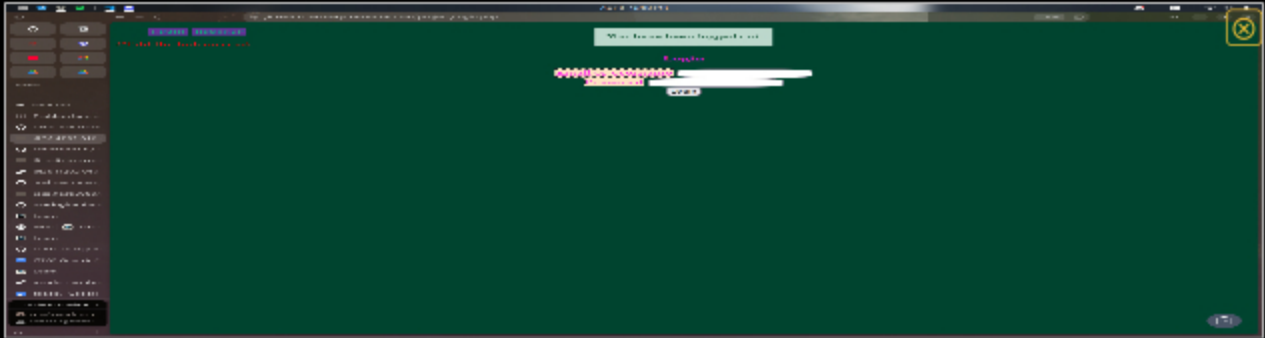
Progress: 100%

#### Part 1:

Progress: 100%

### Details:

- Show the successful logout message (visit the proper file on Render.com qa after a manual deploy)
- Show the code snippet of the logout page



successful logout message

```
1 <?php
2 // require functions.php to pull in flash()
3 require(_DIR_ . "../lib/functions.php");
4 reset_session(); // clear session data and start a new session
5 flash("You have been logged out","success");
6 header("Location: $BASE_PATH/login.php"); // redirect back to login
7 //jlt36 4/8/26 You, now * Uncommitted changes
```

logout code snippet

 Saved: 4/8/2026 11:01:00 PM

## Part 2:

Progress: 100%

### Details:

- Include the pull request url for this feature

URL #1  
<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/18>



URL  
<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/18>

 Saved: 4/8/2026 11:01:00 PM

## Part 3:

Progress: 100%

### Details:

- Briefly explain how the logout process works and how the user is officially logged off

### Your Response:

The logout page clears the currently active session data so the user isn't authenticated anymore. After the session is reset, a success message is shown and the user is directed to the login page.



Saved: 4/8/2026 11:01:00 PM

## Section #4: ( 2 pts.) Feature: Security Rules

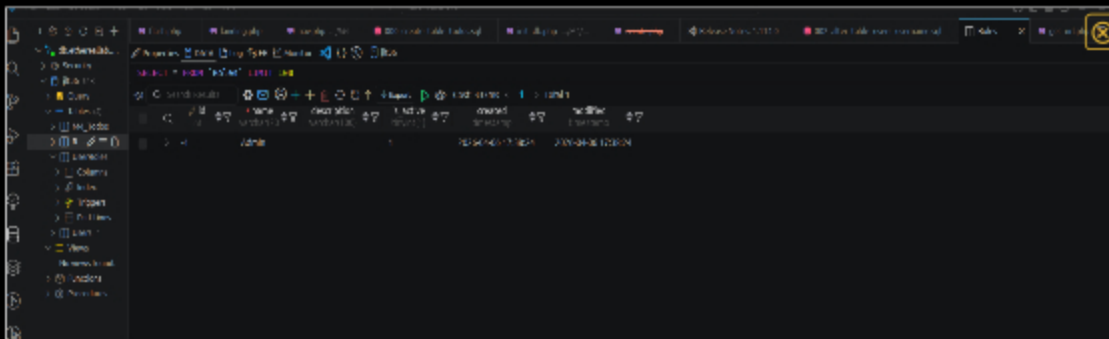
Progress: 100%

### Task #1 ( 1 pt.) - Database Tables

Progress: 100%

#### Details:

- From the vs code mysql tool, show both the `Roles` and `UserRoles` tables with



roles



userroles



Saved: 4/8/2026 12:18:10 PM

### Task #2 ( 1 pt.) - Demo Security Rules

Progress: 100%

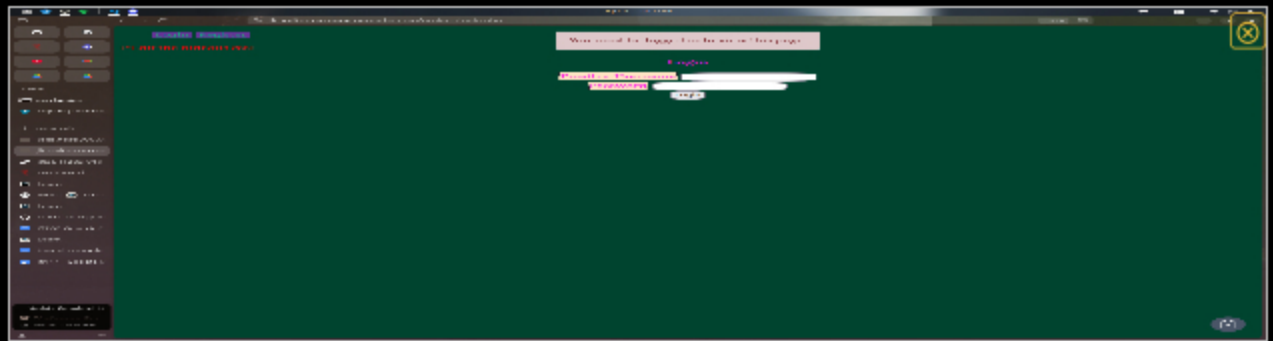
#### Part 1:

Progress: 100%

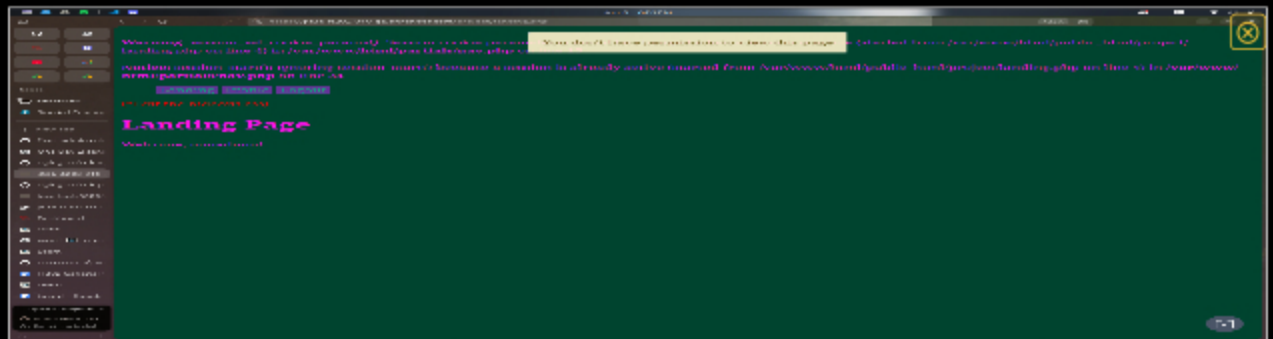
#### Details:

- Show the message related to needing to be logged in (i.e., try to manu
- Show the message related to not having the proper permission/role (i.e
- Include a snippet of the login check function

- Include a snippet of the role check function
- Include a snippet of the role check function



trying to access the landing page or profile page, it redirects to the login page with the message "you must be logged in"



when im logged in as nonadmin and try to access list\_roles/assign\_roles/create\_roles i stay on landing with a pop up msg

```
1 <?php
2 require_once( __DIR__ . "/../lib/functions.php");
3 session_start();
4
5 if (!is_logged_in()) {
6     flash("You must be logged in to view this page.", "danger");
7     header("Location: $BASE_PATH/login.php");
8     exit;
9 }
10 //lt36 4/8/26 You now have uncommitted changes
11 ?>
12 <?php
```

profile.php code snippet

```
1 <?php
2 require( __DIR__ . "/../lib/functions.php");
3
4 session_start();
5
6 if (!is_logged_in()) {
7     flash("You must be logged in to view this page.", "danger");
8     header("Location: /project/login.php");
9     exit;
10 }
11 //lt36 4/8/26
12 error_log("Session: " . var_export($_SESSION, true));
13 ?>
14 <h1>Landing Page</h1>
15
```

landing.php code snippet

```
1 <?php
2 //note we need to go up 1 more directory
3 require_once( __DIR__ . "/../lib/functions.php");
4
```



```

4 if (session_status() === PHP_SESSION_NONE) {
5     session_start();
6 }
7
8 if (!has_role("Admin")) {
9     flash("You don't have permission to view this page", "warning");
10    header("Location: /project/landing.php");
11    exit;
12 } <- #8-12 if (!has_role("Admin"))
13 //jlt36 4/8/26 You, now • Uncommitted changes
14

```

role validation snippet (it is the same for all three)



Saved: 4/8/2026 11:04:46 PM

## ↔ Part 2:

Progress: 100%

### Details:

- Include the pull request url for this feature

URL #1

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/31>



URL

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/31>



URL #2

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/39>



URL

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/39>



Saved: 4/8/2026 11:04:46 PM

## ≡ Part 3:

Progress: 100%

### Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

the login check code verifies whether a valid user session exists before allowing access to a protected page. If no user is logged in, there's a flash msg and it redirects the user to the login page, and stops the script.

the role check code verifies whether the logged in user has the required role (admin) before allowing access to a restricted page. If the user doesn't have the proper role, there's a flash message and redirects the user to a safe page, and stops the script.



Saved: 4/8/2026 11:04:46 PM

# Section 5. ( 2 pts.) Feature: User Profile

Progress: 100%

## ≡ Task #1 ( 1 pt.) - Validation

Progress: 100%

### Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

## ≡ Task #1 ( 0.33 pts.) - HTML Form/Validation

Progress: 100%

### Details:

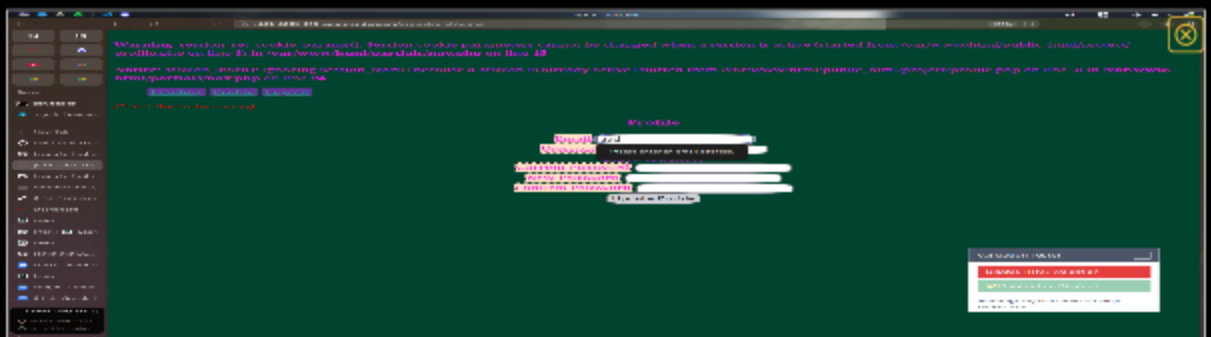
- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

### Part 1:

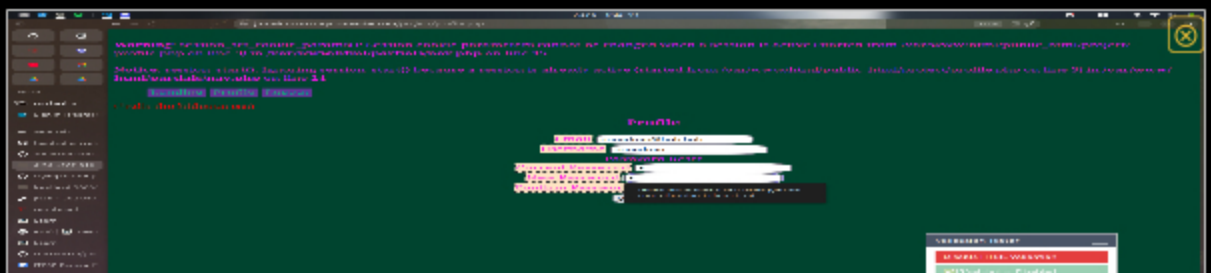
Progress: 100%

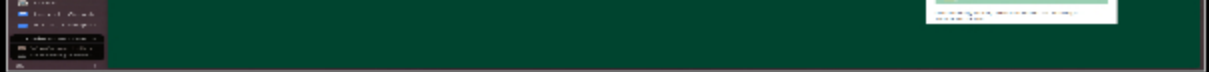
### Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

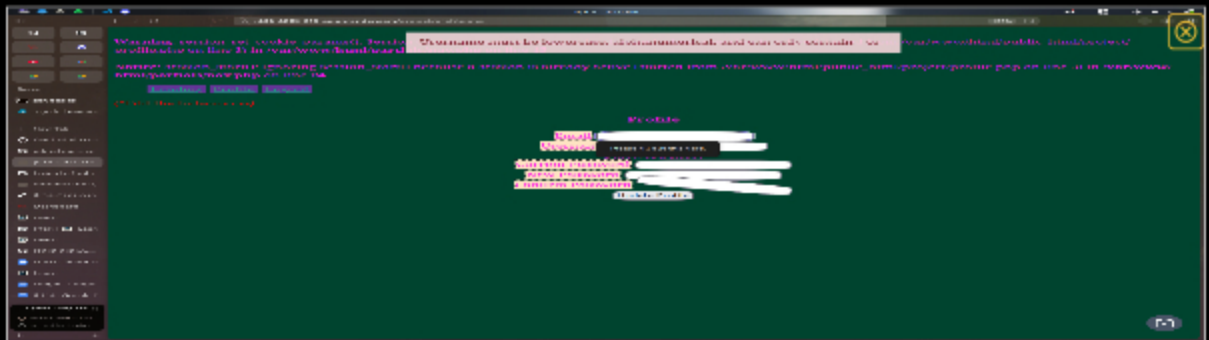


please enter email address

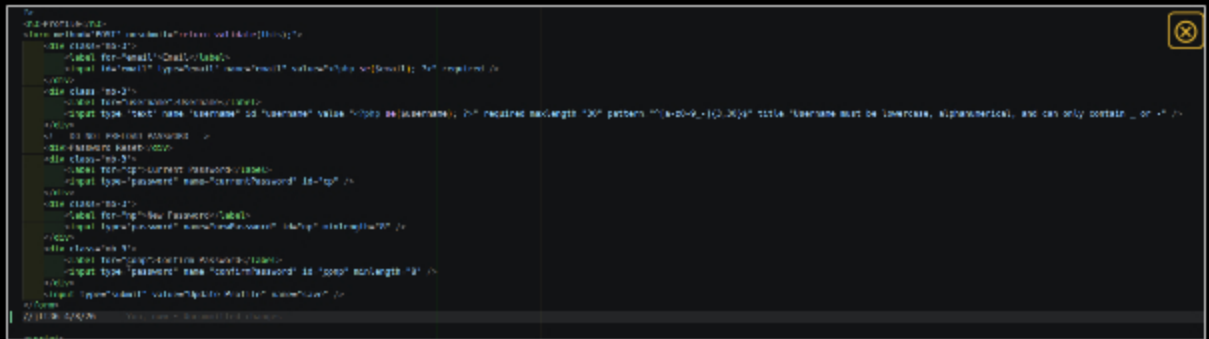




please use at least 8 characters



empty field



html validation snippet



 Saved: 4/8/2026 11:06:11 PM

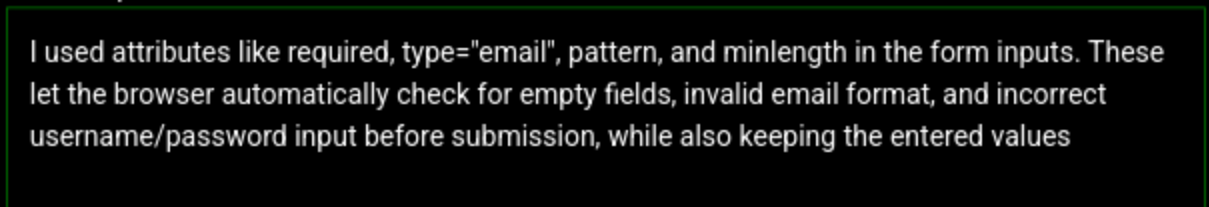
 **Part 2:**

Progress: 100%

**Details:**

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:



## Task #2 ( 0.33 pts.) - JS Validation (validate() function)

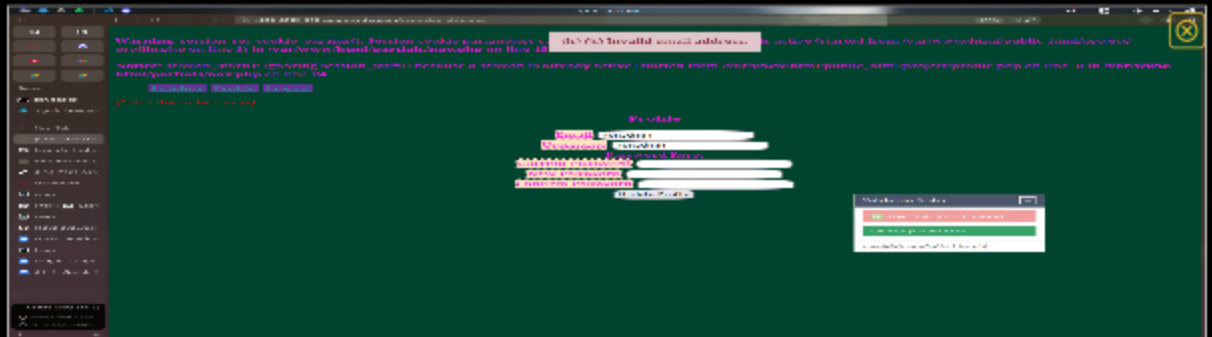
Progress: 100%

 **Part 1:**

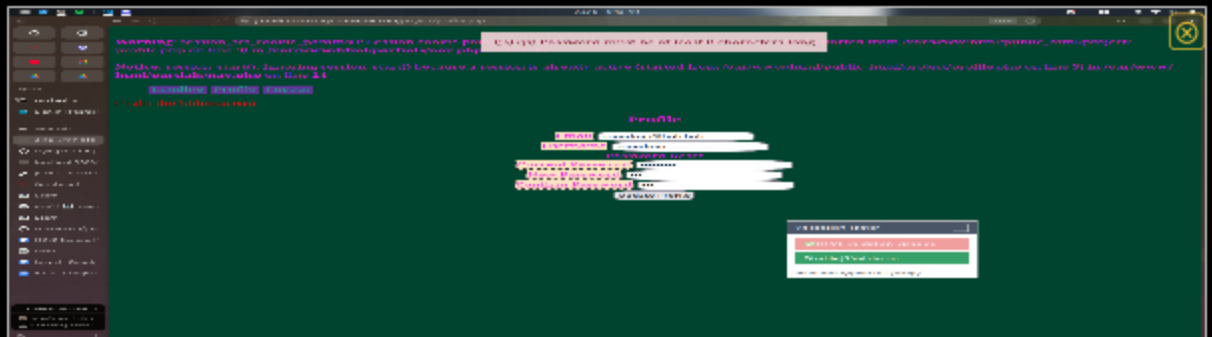
Progress: 100%

**Details:**

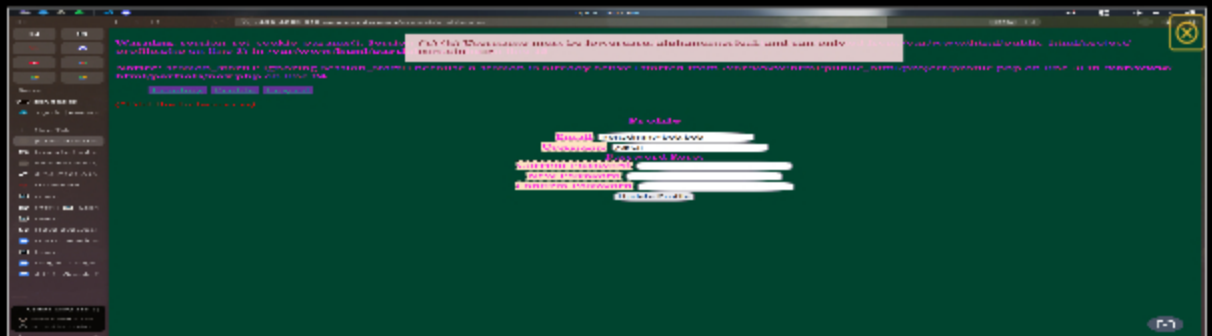
- Show the code related to the profile page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply noValidate to the form tag



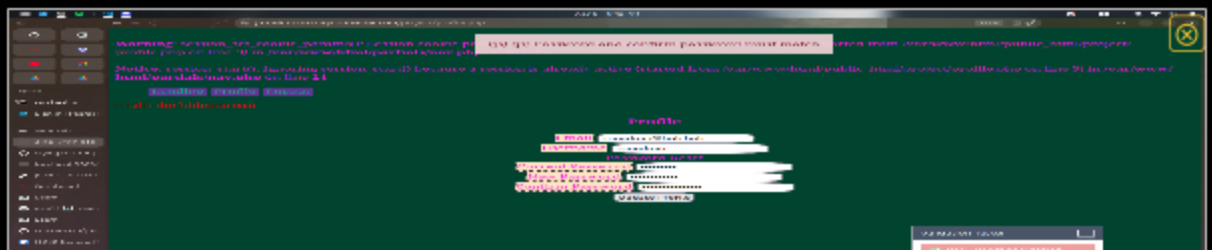
invalid email

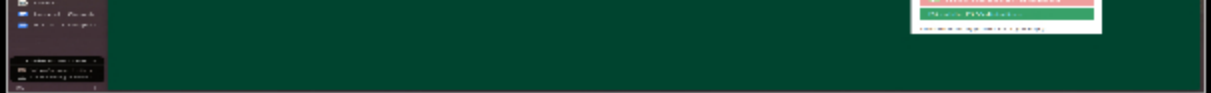


password must be at least 8



username must be lowercase





password and confirm must match

```
//script
function validate(form) {
  let email = form.email.value.trim();
  let username = form.username.value.trim();
  let pw = form.password.value;
  let cpw = form.confirmPassword.value;
  let isValid = true;
  //add other client side validation...

  let usernamePattern = /^[a-zA-Z0-9_]{2,20}$/;
  let emailPattern = /^[a-zA-Z0-9]{1,30}@([a-zA-Z0-9]{1,30})$/;

  //example of using flash via javascript
  //find the flash container's container element, appendChild
  //NOTE: we'll extract the flash code to a function later
  if (typeof window !== 'undefined') {
    let flashContainer = document.getElementById('flash');
    if (flashContainer) {
      //if the container exists, we can use it to display messages
      let flashMessage = 'danger';
      if (usernamePattern.test(username)) {
        flashMessage = 'danger';
      }
      if (emailPattern.test(email)) {
        flashMessage = 'danger';
      }
      if (pw.length < 8 || pw.length > 30) {
        flashMessage = 'danger';
      }
      if (pw !== cpw) {
        flashMessage = 'danger';
      }
      if (flashMessage) {
        //if the flash message is not empty, we can use it to display messages
        flashContainer.innerHTML += `<div class="flash ${flashMessage}">${flashMessage}</div>`;
      }
    }
  }
  //return true will prevent the form from submitting
  return isValid;
}
//add this function to validate(form)
```

js validation code snippet

 Saved: 4/8/2026 11:07:13 PM

## Part 2:

Progress: 100%

### Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

js validation is handled by the validate(form) function that runs when the form is submitted. it checks the username format, email format, password length, and whether the password matches the confirm password, and prevents submission if any of these are invalid.

 Saved: 4/8/2026 11:07:13 PM

## Task #3 ( 0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

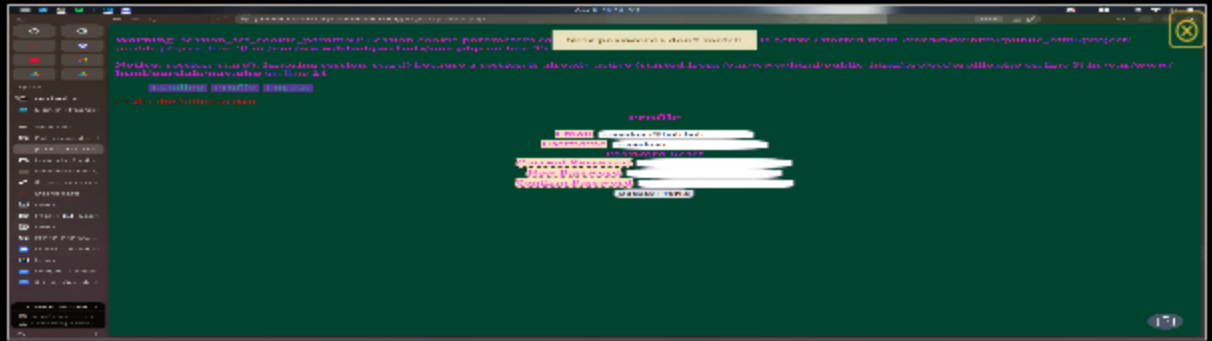
## Part 1:

Progress: 100%

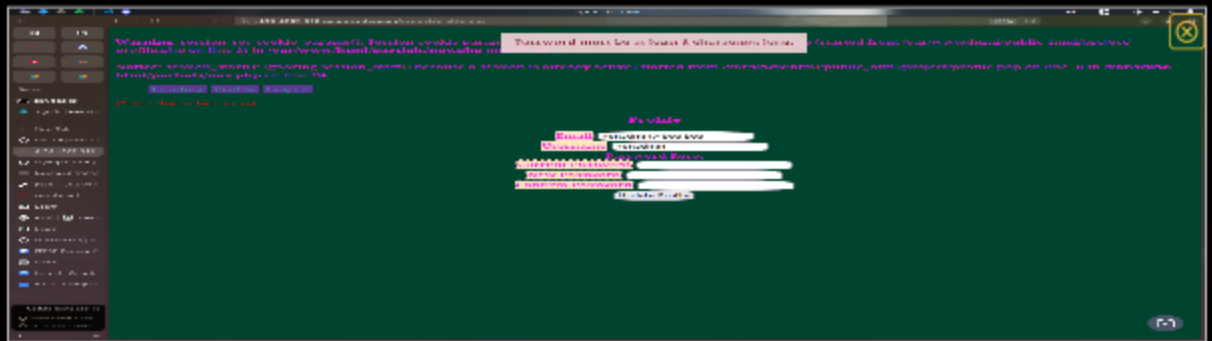
### Details:

- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

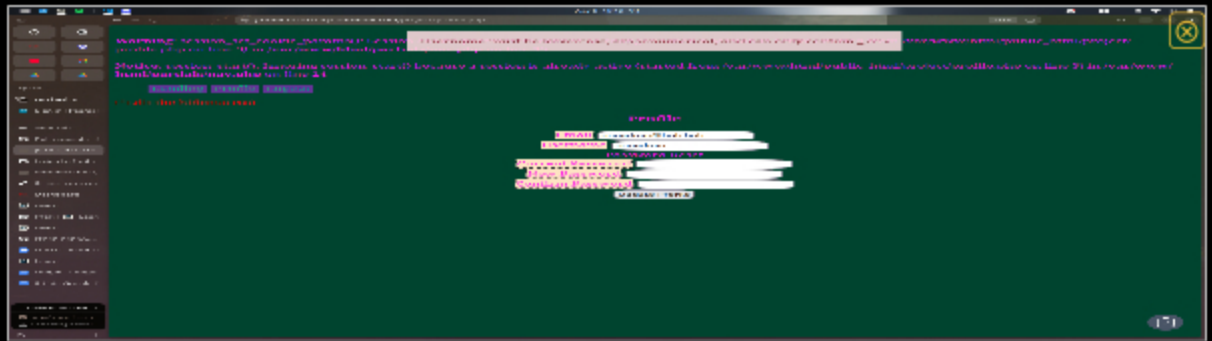
• Note: You may need to use dev tools to temporarily apply novalidate to the form tag and temporarily override the validate() function or use the provided helper script in the instructions



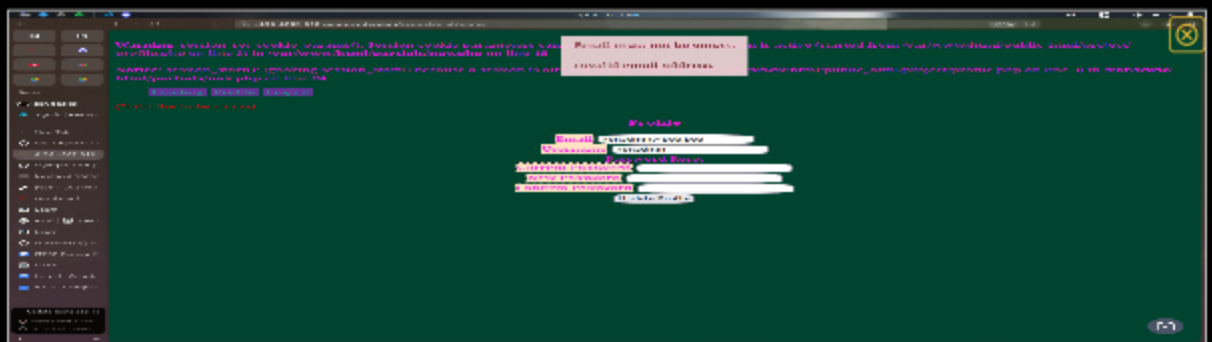
mismatching password



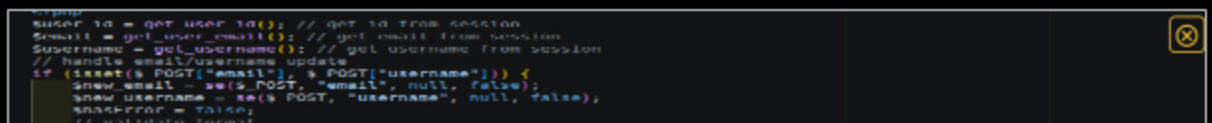
password must be 8 characters



username must be lowercase



email must not be empty and invalid email





```

if(empty($new_email)) {
    //echo "Email must not be empty."; "danger";
    $hasError = true;
}
// 255-255 if (empty($new_email))
// SANITIZE AND VALIDATE email
$new_email = sanitize_email($new_email);
if (!is_valid_email($new_email)) {
    //echo "Invalid email address."; "danger";
    $hasError = true;
}
// 255-255 if (is_valid_email($new_email))
if (!is_valid_username($new_username)) {
    //echo "Username must be lowercase, alphanumerical, and can only contain _ or -. "danger";
    $hasError = true;
}
}
//JUNE 4/8/26 You now have 100 completed changes
//check for changes

```

php email/username validation code snippet

```

//JUNE 4/8/26 You now have 100 completed changes
//check for changes

```

password validation code snippet



Saved: 4/8/2026 11:10:19 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

php validation is handled after the form is submitted by checking the \$\_POST data. It verifies that required fields are filled, the email and username are valid, passwords match and meet length requirements, and current password is correct



Saved: 4/8/2026 11:10:19 PM

Task #2 ( 1 pt.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in .php, , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature



<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/40>



URL

<https://github.com/Joshua-E>



URL #2

[https://github.com/Joshua-Emman/jlt36-IT202-010/blob/qa/public\\_html/project/profile.php](https://github.com/Joshua-Emman/jlt36-IT202-010/blob/qa/public_html/project/profile.php)



URL

<https://github.com/Joshua-E/>



URL #3

[https://jlt36-it202-010-qa.onrender.com/  
project/profile.php](https://jlt36-it202-010-qa.onrender.com/project/profile.php)



URL

<https://jlt36-it202-010-qa.onr>



 Saved: 4/8/2026 9:32:28 PM

## Section #6: ( 1 pt.) Misc

Progress: 100%

### Task #1 ( 0.33 pts.) - Github Details

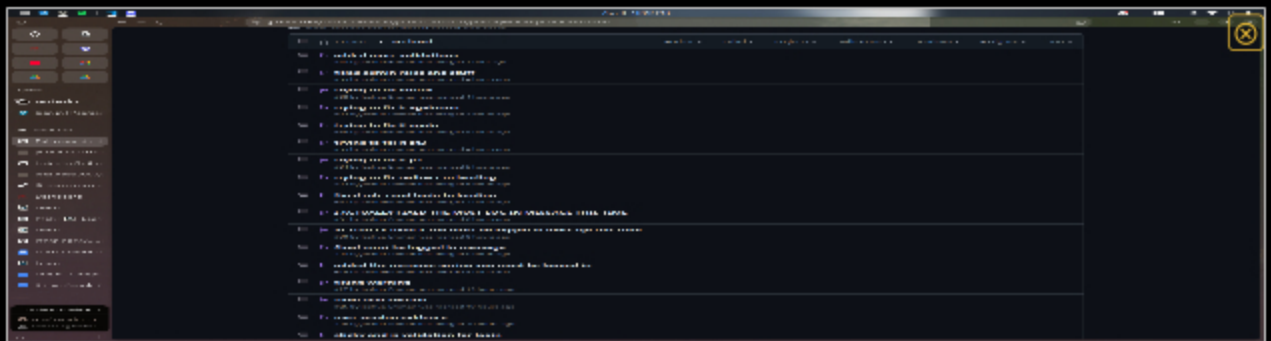
Progress: 100%

## Part 1:

Progress: 100%

### Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)



## my PRs and commits



 Saved: 4/8/2026 10:25:29 PM

## ➡ Part 2:

Progress: 100%

### Details:

Include the link to the Pull Request (should end in /pull/#) (Milestone1 into qa)

URL #1

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/40>



URL

<https://github.com/Joshua-Emma>



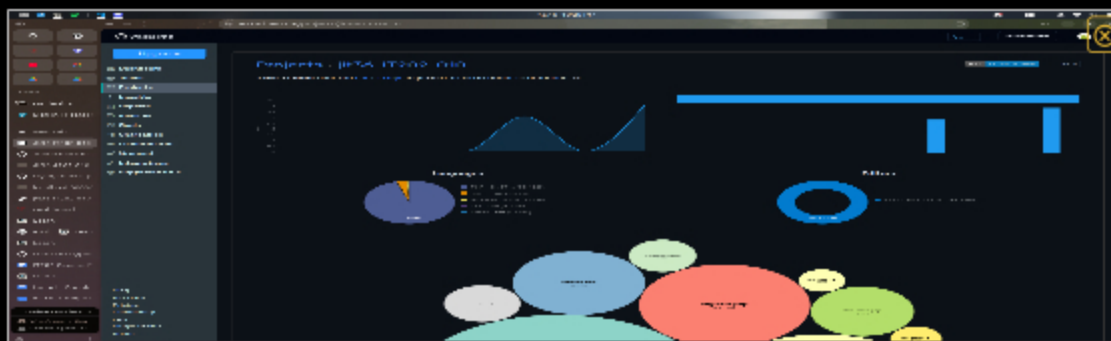
Saved: 4/8/2026 10:25:29 PM

## Task #2 ( 0.33 pts.) - WakaTime - Activity

Progress: 100%

### Details:

- Visit the WakaTime.com Dashboard
- Click Projects and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



top wakatime



bottom wakatime



Saved: 4/8/2026 10:29:06 PM

## Task #3 ( 0.33 pts.) - Reflection

Progress: 100%

– Task #1 ( 0.33 pts.) – What did you learn?

### ⇒ Task #1 ( 0.33 pts.) - What did you learn?

Progress: 100%

#### Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how to use different types of validation like HTML, js, and PHP. Each one does something different, like HTML checks basic stuff, js gives quick feedback, and PHP makes sure everything is valid on the backend. I also learned how to keep form data from getting cleared so users don't have to retype anything.



Saved: 4/8/2026 10:41:13 PM

### ⇒ Task #2 ( 0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

#### Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

Easiest part was probably adding HTML validation. It was pretty simple to just add things like required and type="email" to the inputs. The browser basically handles everything.



Saved: 4/8/2026 10:42:16 PM

### ⇒ Task #3 ( 0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

#### Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part was definitely the debugging errors and making sure everything was working correctly. I had a lot of trouble figuring out why everything was working on localhost and not on render.



Saved: 4/8/2026 10:43:53 PM